



Universidad de las Fuerzas Armadas ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería en Software

Desarrollo de Software Seguro

Proyecto Integrador Parcial III

Docente: Geovanny Cudco

30 de junio de 2026

1. Tema

Sistema de Autenticación y Autorización Centralizado (Master Gateway)

2. Descripción General:

En la actualidad, el desarrollo de aplicaciones empresariales ha evolucionado hacia arquitecturas de microservicios para garantizar escalabilidad, independencia de despliegue y diversidad tecnológica. Sin embargo, esta descentralización trae consigo un desafío crítico: la fragmentación de la identidad y el control de acceso.

Cuando cada microservicio implementa su propio mecanismo de autenticación y autorización, se generan silos de seguridad, duplicación de código, inconsistencias en la gestión de roles y una experiencia de usuario fragmentada. Además, la integración de nuevos módulos al ecosistema se vuelve lenta y propensa a errores de configuración, exponiendo vulnerabilidades críticas (como Broken Access Control).

El problema fundamental radica en la falta de un "Microservicio Maestro (Master)" que actúe como el eje centralizador de la identidad. Se necesita un sistema que no solo valide "quién" es el usuario (Autenticación), sino "qué" puede hacer y "dónde" puede ir (Autorización dinámica), basándose en una estructura de menús recursiva que se adapte intrínsecamente a los módulos asignados a sus roles.

Para solventar esto, el proyecto exige la construcción de un módulo Full-Stack bajo los principios de Shift-Left (integración de seguridad desde las primeras fases de diseño y codificación) y Zero Trust ("Nunca confiar, siempre verificar"; asumiendo que la red interna es hostil y requiriendo validación continua).

3. Objetivo General

Desarrollar un microservicio maestro de autenticación y autorización *Full-Stack* que centralice la gestión de identidades, roles y navegación, sirviendo como *gateway* de seguridad y enrutamiento para un ecosistema de microservicios.

3.1. Objetivos Específicos

- OE 1** Implementar un modelo de datos relacional que soporte relaciones *Many-to-Many* entre **Usuarios** y **Roles**.
- OE 2** Diseñar una estructura de menús dinámica y recursiva (almacenada en una sola tabla) que represente **Módulos**, **Submenús** e **Items**, asociada directamente a roles.
- OE 3** Desarrollar el flujo de inicio de sesión donde el usuario seleccione activamente el rol con el que desea operar, cargando únicamente el módulo y menú asociados a dicha elección.
- OE 4** Configurar una arquitectura preparada para la integración de futuros microservicios (ej. Módulo de Ventas), donde el microservicio maestro emita y valide tokens (*JWT/OAuth2*) bajo el modelo *Zero Trust*.
- OE 5** Aplicar el enfoque *Shift-Left* mediante pruebas de seguridad unitarias, validación de entradas (sanitización), protección contra inyección SQL (vía ORM) y cifrado de contraseñas.

4. Arquitectura y Modelo de Datos

4.1. Modelo Entidad-Relación

A continuación se describen las principales relaciones que rigen el modelo de datos del microservicio maestro:

Usuarios $\longleftrightarrow$ Roles

Relación **M:N** — Un usuario puede tener múltiples roles asignados y, a su vez, un rol puede estar asignado a múltiples usuarios. Esta relación se materializa mediante una tabla intermedia (*join table*) que almacena los pares `usuario_id` – `rol_id`.

Roles $\longleftrightarrow$ Módulos

Relación **1:N** o **M:N** — Un rol puede tener acceso a uno o varios módulos administrativos. Si se requiere granularidad fina (por ejemplo, un módulo compartido por varios roles con distintos permisos), se adopta el modelo **M:N** con tabla intermedia; de lo contrario, basta con una clave foránea `rol_id` en la tabla de módulos.

Menús (Estructura Recursiva)

Se emplea una **única tabla** Menu definida por los siguientes campos:

Campo	Descripción
id	Identificador único del registro.
nombre	Etiqueta visible en la interfaz (ej. <i>Ventas</i> , <i>Reportes</i>).
url	Ruta hacia el microservicio destino. Solo se completa en nodos hoja (Items).
modulo_id	Clave foránea que vincula el menú con su módulo padre.
parent_id	Referencia al id de otro registro de la misma tabla. Si es NULL, el nodo es un Menú Principal ; si tiene valor, es un Submenú o Item .

De esta forma, únicamente los **nodos hoja** (aquellos que no tienen hijos) contendrán el enlace (`url`) hacia el nuevo microservicio, mientras que los nodos intermedios actúan como agrupadores jerárquicos.

4.2. Uso de Tecnologías

Frameworks

Se deberán utilizar **frameworks modernos y robustos** que faciliten la creación de estructuras seguras y modulares. Algunas opciones recomendadas son:

- **NestJS** — Basado en TypeScript, con decoradores, inyección de dependencias y módulos nativos de seguridad (*guards*, *interceptors*, *pipes*).
- **Spring Boot** — Ecosistema Java maduro con *Spring Security* para autenticación/autorización declarativa.
- **FastAPI** — Framework Python de alto rendimiento con validación automática mediante Pydantic y soporte nativo para *OAuth2*.
- **Django** — Incluye un sistema de autenticación integrado y middleware extensible para lógica de seguridad.

ORMs

Es **obligatorio** el uso de un *Object-Relational Mapper* (ORM) con el fin de:

1. **Abstraer la base de datos:** Evitar escribir consultas SQL en bruto, logrando independencia del motor subyacente (PostgreSQL, MySQL, etc.).

2. **Prevenir inyección SQL:** Todas las consultas se construyen mediante parámetros vinculados internamente por el ORM, eliminando la posibilidad de inyección.
3. **Facilitar consultas recursivas:** Permitir la ejecución de *Common Table Expressions* (CTE) generadas automáticamente por el ORM para recorrer la estructura jerárquica de la tabla `Menu`.

Algunas opciones recomendadas de ORMs compatibles con los frameworks mencionados:

Ecosistema	ORM Sugerido
Node.js / NestJS	Prisma, TypeORM
Java / Spring Boot	Hibernate, Spring Data JPA
Python / FastAPI	SQLAlchemy, Tortoise ORM
Python / Django	Django ORM (integrado)
.NET	Entity Framework Core

5. Requisitos Funcionales Específicos

5.1. Gestión de Identidad

CRUD completo para **Usuarios** y **Roles**, manejando adecuadamente la **tabla pivote intermedia** que materializa la relación M:N entre ambas entidades. Todas las operaciones de creación, lectura, actualización y eliminación deben garantizar la integridad referencial y la consistencia transaccional.

5.2. Gestión de Módulos y Menús

- **Creación de Módulos:** Registro de unidades funcionales del sistema (ej. *Ventas*, *Recursos Humanos*, *Financiero*).
- **Asignación de Módulos a Roles:** Vinculación que determina qué roles tienen visibilidad sobre qué módulos administrativos.
- **CRUD de Menús con patrón Adjacency List:** Implementación del menú en una **única tabla** utilizando el patrón *Adjacency List* (lista de adyacencia), donde cada registro referencia a su nodo padre mediante `parent_id`, soportando así la recursividad de forma eficiente sin necesidad de múltiples tablas.

5.3. Pantalla de “Espacio de Trabajo” (*Workspace Selector*)

Posterior al *login* clásico (credenciales válidas), el sistema **debe impedir** la carga directa del *dashboard*. En su lugar, se debe **forzar la selección explícita del rol** con el que el usuario desea operar en esa sesión. Esta selección permite:

- Delimitar el **contexto de seguridad** de toda la sesión.
- Implementar *Tenant/Rol Isolation* a nivel de sesión, garantizando que cada instancia de trabajo esté aislada y acotada exclusivamente a los permisos del rol seleccionado.

5.4. Enrutamiento Basado en Menú

El **Frontend** *no debe tener las rutas **hardcodeadas***. En su lugar, debe construirlas **dinámicamente** a partir del JSON del menú devuelto por el microservicio *Master* tras la selección del rol. Esto implica que la estructura de navegación completa —módulos, submenús e ítems— es determinada en tiempo de ejecución por el backend, eliminando cualquier dependencia estática del cliente.

6. Requisitos No Funcionales y de Seguridad (*Shift-Left & Zero Trust*)

6.1. *Zero Trust Architecture (ZTA)*

- **Validación obligatoria en cada endpoint:** Todos los endpoints del sistema deben requerir un token válido; no existen rutas o recursos de acceso público una vez autenticado el flujo.
- **Delegación de confianza al Master:** Los microservicios hijos (futuros, ej. Módulo de Ventas) **no deben tener su propia base de datos de usuarios**. Deben cumplir una de las siguientes estrategias:
 - a. *Validación directa:* Llamar al microservicio Master para validar el token en cada petición.
 - b. *Validación asimétrica (opcional):* Compartir una clave criptográfica asimétrica (par pública/privada) que permita al microservicio hijo validar la firma del JWT **sin necesidad de comunicación directa** con el Master, reduciendo latencia y acoplamiento.

6.2. Principio de Menor Privilegio (*Least Privilege*)

El token generado al seleccionar el rol **solo debe contener los permisos estrictamente necesarios** para ese rol. No se incluyen permisos globales del usuario, ni de otros roles que pudiera tener asignados. Esto minimiza el impacto en caso de compromiso del token.

6.3. *Shift-Left Security*

- **Análisis estático de código (SAST):** Integrado en el pipeline de CI/CD **desde el primer día** de desarrollo, detectando vulnerabilidades antes de que el código llegue a producción.
- **Consultas parametrizadas exclusivas:** Todo acceso a datos debe realizarse **exclusivamente** a través del ORM con consultas parametrizadas. Queda **prohibida** la concatenación de cadenas (*string interpolation*) para construir consultas SQL.
- **Hash de contraseñas robusto:** Almacenamiento de contraseñas utilizando algoritmos de *hash* lentos y adaptativos, tales como **Argon2** o **Bcrypt** con un alto factor de costo (*cost factor*), resistente a ataques de fuerza bruta.

- **Gestión segura de Secrets:** Todas las credenciales, claves y tokens deben almacenarse mediante **variables de entorno** o gestores de secrets (ej. *Vault*, *AWS Secrets Manager*). Queda **totalmente prohibido** el *hardcodeo* de secrets en el código fuente.

6.4. *Performance*

Las consultas recursivas del menú deben estar **optimizadas** para evitar el problema clásico de **consultas N+1**. Se deberán preferir técnicas de carga en árbol proporcionadas por el ORM o el uso de *Common Table Expressions* (CTE) nativas de la base de datos, garantizando tiempos de respuesta predecibles independientemente de la profundidad del menú.

7. Consideraciones para el Desarrollo

7.1. Base de Datos

Se sugiere emplear **PostgreSQL** o **MySQL**, dado que ambos motores ofrecen **soporte nativo y eficiente** para consultas jerárquicas recursivas (CTE con la cláusula `WITH RECURSIVE` en PostgreSQL; CTE recursivas disponibles desde MySQL 8.0), lo cual es fundamental para la tabla de menús con patrón *Adjacency List*.

7.2. Estado de la Sesión

Se optará por una arquitectura ***Stateless*** en el servidor mediante el uso de tokens **JWT** (*JSON Web Tokens*). Esta decisión garantiza que el microservicio Master **no colapse bajo carga** a medida que se añadan más microservicios al ecosistema, ya que no es necesario mantener estado de sesión en memoria ni en base de datos del lado del servidor.

7.3. Frontend

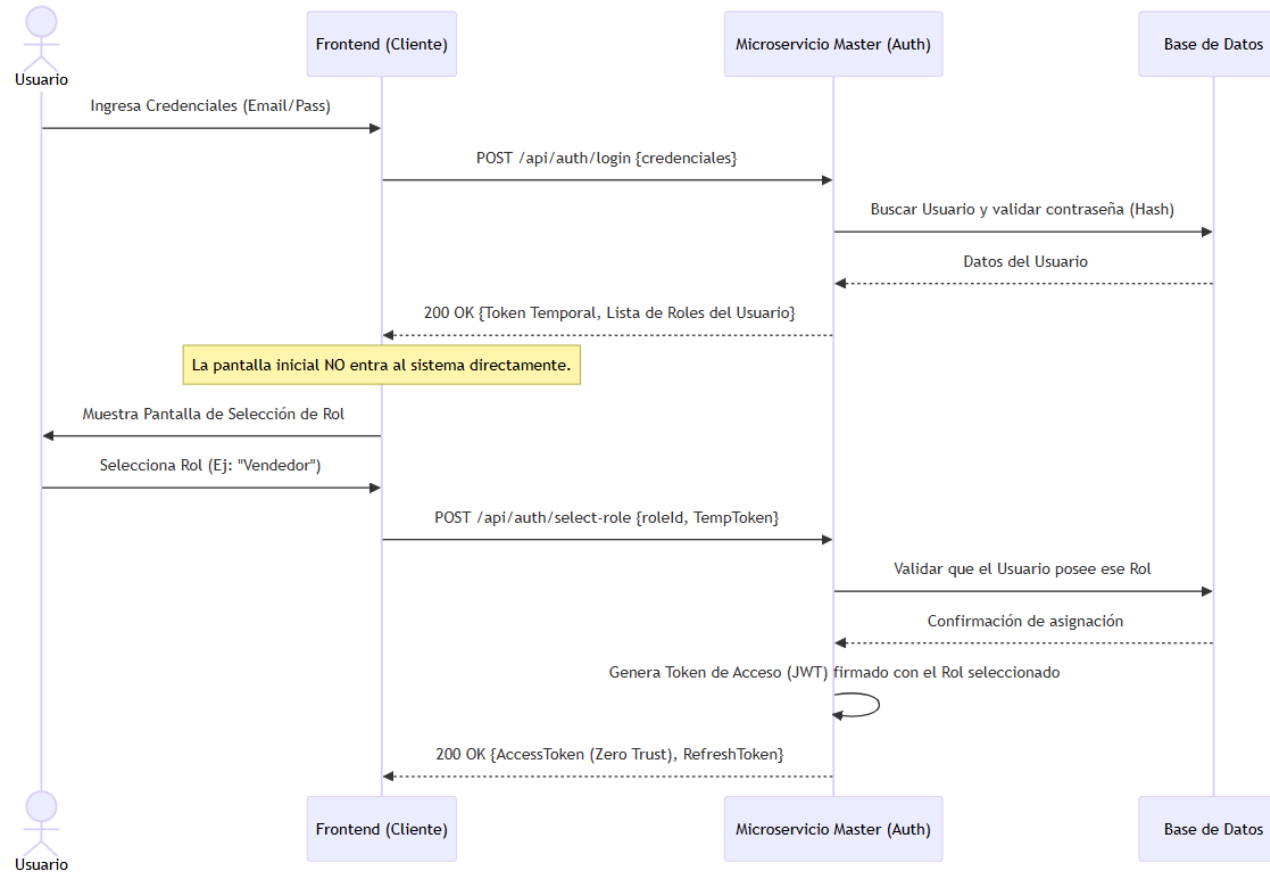
El cliente debe ser una **SPA** (*Single Page Application*) capaz de:

- **Interceptar** las respuestas de la API del menú provenientes del Master tras la selección del rol.
- **Inyectar rutas dinámicamente** al enrutador del framework utilizado (ej. *Vue Router* para Vue.js, *React Router* para React, *Angular Router* para Angular), de modo que la navegación se construya en tiempo de ejecución sin dependencias estáticas en el código fuente del cliente.

8. Diagramas de Secuencia

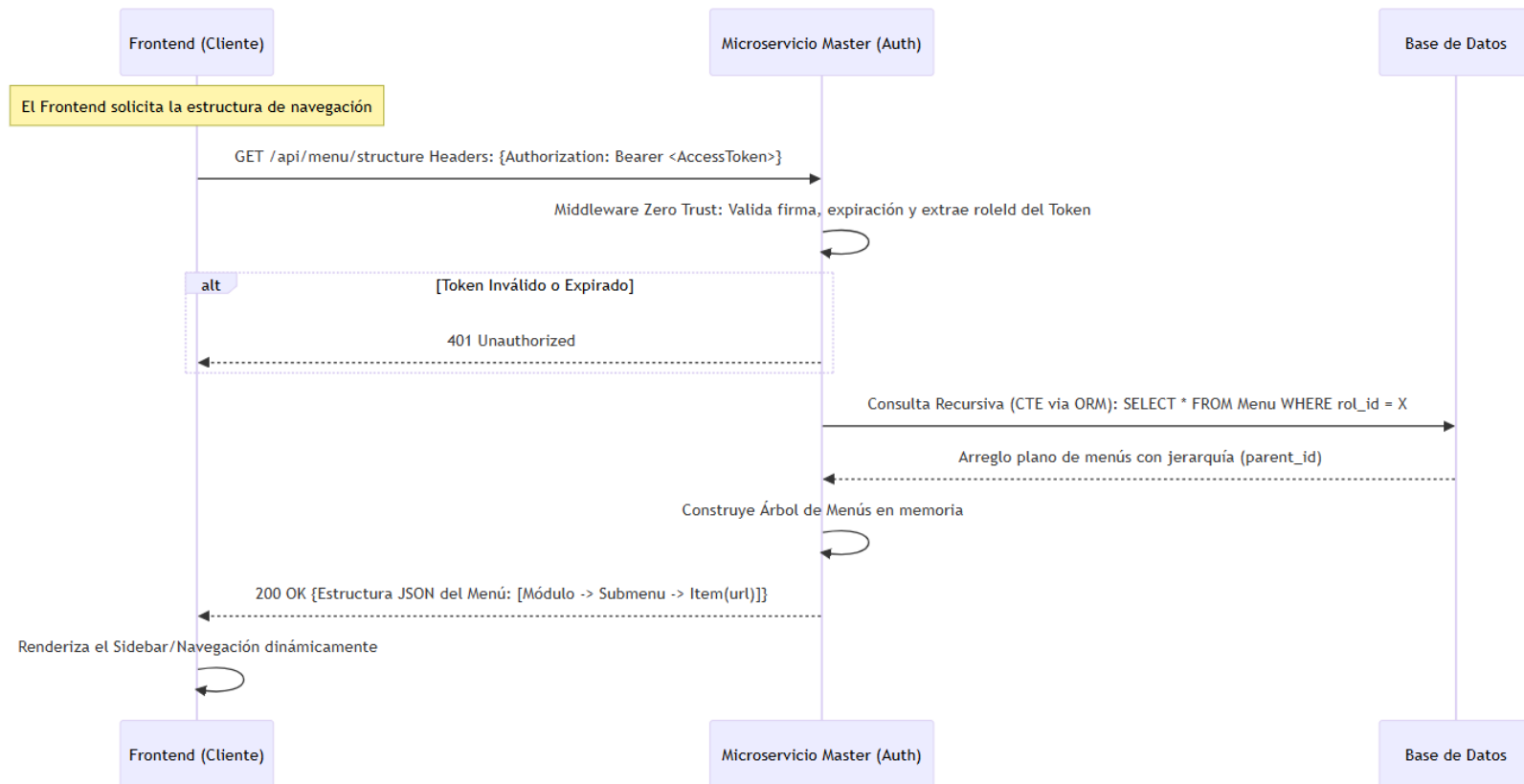
8.1. Flujo de Autenticación y Selección de Rol

Figura 1: Diagrama de secuencia



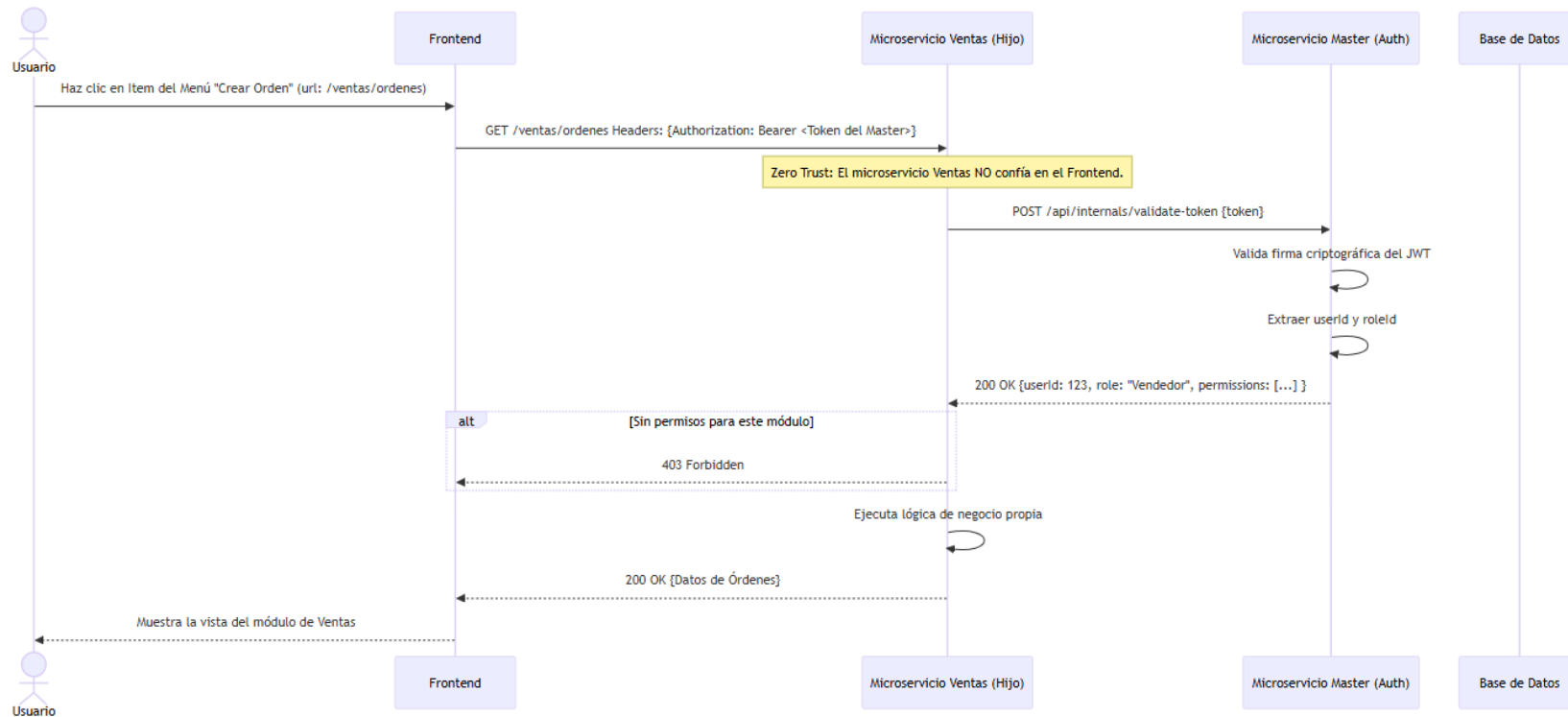
8.2. Flujo de Carga Dinámica del Menú Recursivo

Figura 2: Diagrama de secuencia



8.3. Flujo de Integración con Microservicio Hijo (Ej. Ventas) - Zero Trust

Figura 3: Diagrama de secuencia



9. Especificación de Endpoints del Microservicio Maestro

A continuación, se presenta la especificación de los endpoints mínimos requeridos para el Microservicio Maestro.

Previo a la tabla, es fundamental definir el **Patrón de Auditoría y Estado Global**, el cual aplica de manera estricta para todas y cada una de las entidades de la base de datos (Usuarios, Roles, Módulos, Menús), garantizando trazabilidad y eliminación lógica.

Estándar de Campos Obligatorios por Entidad (ORM)

Cualquier tabla creada a través del ORM debe heredar o incluir los siguientes campos por defecto:

- **id**: UUID o Auto-incremental (Según estrategia DB).
- **estado**: Booleano o Enum (ACTIVO, INACTIVO). **Nunca se debe eliminar físicamente un registro (DELETE duro)**, se debe hacer un *Soft Delete* actualizando este campo a INACTIVO.
- **fecha_creacion**: *Timestamp* automático en el momento de la inserción (Managed by ORM).
- **fecha_actualizacion**: *Timestamp* automático que se actualiza en cada UPDATE (Managed by ORM).
- **creado_por**: UUID del usuario que creó el registro (Null si es auto-registro).
- **actualizado_por**: UUID del usuario que modificó el registro por última vez.

Tabla de Endpoints Mínimos de la Aplicación

Dominio / Recurso	Método HTTP	Endpoint	Descripción	Seguridad / Negocio
AUTENTICACIÓN				
Inicio de Sesión	POST	/api/auth/login	Valida credenciales, devuelve TempToken y lista de roles del usuario.	Rate limiting estricto. No debe revelar si el usuario o la contraseña es el incorrecto (mensaje genérico).
Selección de Rol	POST	/api/auth/select-role	Recibe TempToken y roleId, devuelve JWT definitivo.	El JWT emitido debe tener tiempo de expiración corto (Zero Trust).
Renovar Token	POST	/api/auth/refresh-token	Genera un nuevo JWT usando un Refresh Token válido.	Revocación inmediata si se detecta reutilización de un Refresh Token.
Cerrar Sesión	POST	/api/auth/logout	Invalida los tokens del usuario en la base de datos/Redis.	Necesario para cortar la sesión de inmediato en caso de compromiso.
VALIDACIÓN INTERNA				
Validar Token	POST	/api/internals/validate-token	Endpoint privado para que otros microservicios validen el JWT.	No expone datos sensibles, solo confirma validez, userId y roleId.
USUARIOS				
<i>Continúa en la siguiente página...</i>				

<i>Continuación de la Tabla de Endpoints Mínimos</i>				
Dominio / Recurso	Método HTTP	Endpoint	Descripción	Seguridad / Negocio
Listar Usuarios	GET	/api/users	Obtiene lista paginada de usuarios.	Filtros obligatorios por estado = ACTIVO.
Obtener Usuario	GET	/api/users/{id}	Obtiene detalle de un usuario específico.	Ocultar campo de contraseña hashada en la serialización (ORM).
Crear Usuario	POST	/api/users	Registra un nuevo usuario (Hash de contraseña).	Validación fuerte de contraseña (Shift-Left). El ORM asigna fecha_creacion.
Actualizar Usuario	PUT	/api/users/{id}	Actualiza datos de un usuario.	El ORM actualiza automáticamente fecha_actualizacion y actualizado_por.
Eliminar Usuario	DELETE	/api/users/{id}	Eliminación lógica del usuario.	NO borra el registro , cambia estado a INACTIVO.
ROLES				
Listar Roles	GET	/api/roles	Obtiene todos los roles activos.	—
<i>Continúa en la siguiente página...</i>				

<i>Continuación de la Tabla de Endpoints Mínimos</i>				
Dominio / Recurso	Método HTTP	Endpoint	Descripción	Seguridad / Negocio
Crear Rol	POST	/api/roles	Crea un nuevo rol en el sistema.	—
Actualizar Rol	PUT	/api/roles/{id}	Modifica nombre/descripción del rol.	—
Eliminar Rol	DELETE	/api/roles/{id}	Eliminación lógica del rol.	Prevenir eliminación si está asignado a usuarios activos.
Asignar Rol a Usuario	POST	/api/roles/{id}/users	Asocia un usuario existente a este rol (M:N).	Registra en la tabla pivote con sus propios campos de auditoría.
Desasignar Rol	DELETE	/api/roles/{id}/users/{userId}	Rompe la relación M:N (Eliminación física en la tabla pivote).	—
MÓDULOS				
Gestionar Módulos	GET POST PUT DELETE	/api/modules /api/modules/{id}	CRUD estándar para los módulos administrativos (Ej: “Ventas”).	Al inactivar un módulo, sus menús asociados no deben renderizarse.
<i>Continúa en la siguiente página...</i>				

Continuación de la Tabla de Endpoints Mínimos				
Dominio / Recurso	Método HTTP	Endpoint	Descripción	Seguridad / Negocio
Asignar Módulo a Rol	POST	/api/roles/{id}/modules	Vincula un módulo completo a un rol.	—
MENÚS (Rekursivos)				
Obtener Árbol de Menú	GET	/api/menus/tree	Devuelve la estructura jerárquica completa basada en el rol del JWT.	Crítico: El ORM debe usar CTE (<i>Common Table Expressions</i>) para resolver la recursividad (<code>parent_id</code>).
Crear Ítem de Menú	POST	/api/menus	Crea Menú, Submenú o Ítem.	Requiere recibir <code>parent_id</code> (null si es raíz) y <code>url</code> (null si no es hoja/item final).
Actualizar Ítem de Menú	PUT	/api/menus/{id}	Modifica texto, url o padre del menú.	Validar que el nuevo <code>parent_id</code> no genere un bucle infinito (referencia cíclica).
Eliminar Ítem de Menú	DELETE	/api/menus/{id}	Eliminación lógica del menú.	Si se elimina un padre, la lógica de frontend/backend debe ignorar los hijos.
Continúa en la siguiente página...				

<i>Continuación de la Tabla de Endpoints Mínimos</i>				
Dominio / Recurso	Método HTTP	Endpoint	Descripción	Seguridad / Negocio
Asignar Menú a Rol	POST	/api/roles/{id}/menus	Asigna un Item/Sub-menú específico a un rol.	—

Notas de Implementación para el Desarrollador

1. **Gestión de Auditoría en el ORM:** En frameworks como NestJS (con TypeORM/Prisma) o Python (con SQLAlchemy), se deben implementar *Hooks* del ciclo de vida del ORM (ej. `@BeforeUpdate`, `@BeforeInsert`) para garantizar que `fecha_actualizacion` y `estado` no puedan ser manipulados manualmente desde el controlador, reforzando el enfoque **Shift-Left**.
2. **Soft Deletes en Consultas:** Todas las entidades deben tener configurado un *Global Scope* o filtro automático a nivel de ORM (ej. `where: { estado: 'ACTIVO' }`) para que un desarrollador no deba escribirlo manualmente en cada endpoint y evitar filtrar datos inactivos por error.
3. **Seguridad en la Tabla Pivote (M:N):** La tabla intermedia (ej. `user_has_roles`) **no es una tabla tonta**; también debe heredar los campos de auditoría (`fecha_creacion`, `estado`) para saber cuándo se le otorgó o revocó un permiso a un usuario específico.

Anexo: Requisitos de Infraestructura, CI/CD y DevSecOps

Estrategia de Ramas (Git Branching Strategy)

El repositorio en GitHub debe regirse estrictamente por el siguiente modelo de ramas:

- **main:** Rama de producción. El código aquí debe ser inmutable excepto mediante *Pull Requests* desde **test**. **Únicamente los merges en esta rama disparan el despliegue automático.**
- **test:** Rama de pruebas/QA. Aquí se integran las funcionalidades para ser validadas por el equipo de calidad o el cliente. Los *Pull Requests* hacia **main** nacen de aquí.
- **dev:** Rama de desarrollo. Los desarrolladores crean ramas *feature* (ej. *feature/auth-login*) a partir de **dev** y hacen *Pull Requests* de vuelta a **dev** para integración continua.

Pipeline CI/CD con GitHub Actions

Se debe configurar un archivo de flujo de trabajo (`.github/workflows/ci-cd.yml`) que ejecute los siguientes pasos de forma secuencial al realizar un *push* o *merge* en la rama **main**:

1. **Build y Pruebas Unitarias:** Compilación del proyecto y ejecución de pruebas.
2. **Análisis Estático Tradicional (SonarCloud):** Integración con SonarCloud para evaluar la calidad del código (Code Smells, Bugs, Vulnerabilidades conocidas y Cobertura de Pruebas). Se debe exigir que el *Quality Gate* pase para continuar.
3. **Análisis SAST Avanzado (Modelo de Minería de Datos/ML):** Ejecución de un contenedor Docker o script de Python que aloje un modelo de Machine Learning pre-entrenado. Este modelo analizará los cambios en el código buscando patrones anómalos o vulnerabilidades lógicas complejas que las reglas estáticas de SonarCloud no detectan.
4. **Despliegue Automático:** Si los pasos 2 y 3 son exitosos, el pipeline ejecutará los comandos de la CLI del proveedor cloud para desplegar la nueva versión.

Infraestructura Cloud (Despliegue Automático)

El microservicio maestro debe desplegarse en un servicio de plataforma como servicio (PaaS) gratuito orientado a desarrolladores, como **Railway** o **Render**.

- **Requisito:** El despliegue debe ser *triggered* por la CLI en el pipeline de GitHub Actions (no solo por webhook automático del repositorio), para asegurar que el código desplegado haya pasado los análisis de seguridad previamente.
- **Gestión de Variables de Entorno:** Las credenciales de base de datos y claves criptográficas (JWT Secret) deben inyectarse desde los *Secrets* de GitHub Actions al entorno del PaaS, nunca almacenarse en el código.

Sistema de Notificaciones (Telegram Bot)

Se debe crear un Bot de Telegram (vía `BotFather`) e integrar su token en los secretos del repositorio. El pipeline debe utilizar la API de Telegram para enviar mensajes a un grupo específico (donde estén todos los integrantes del equipo).

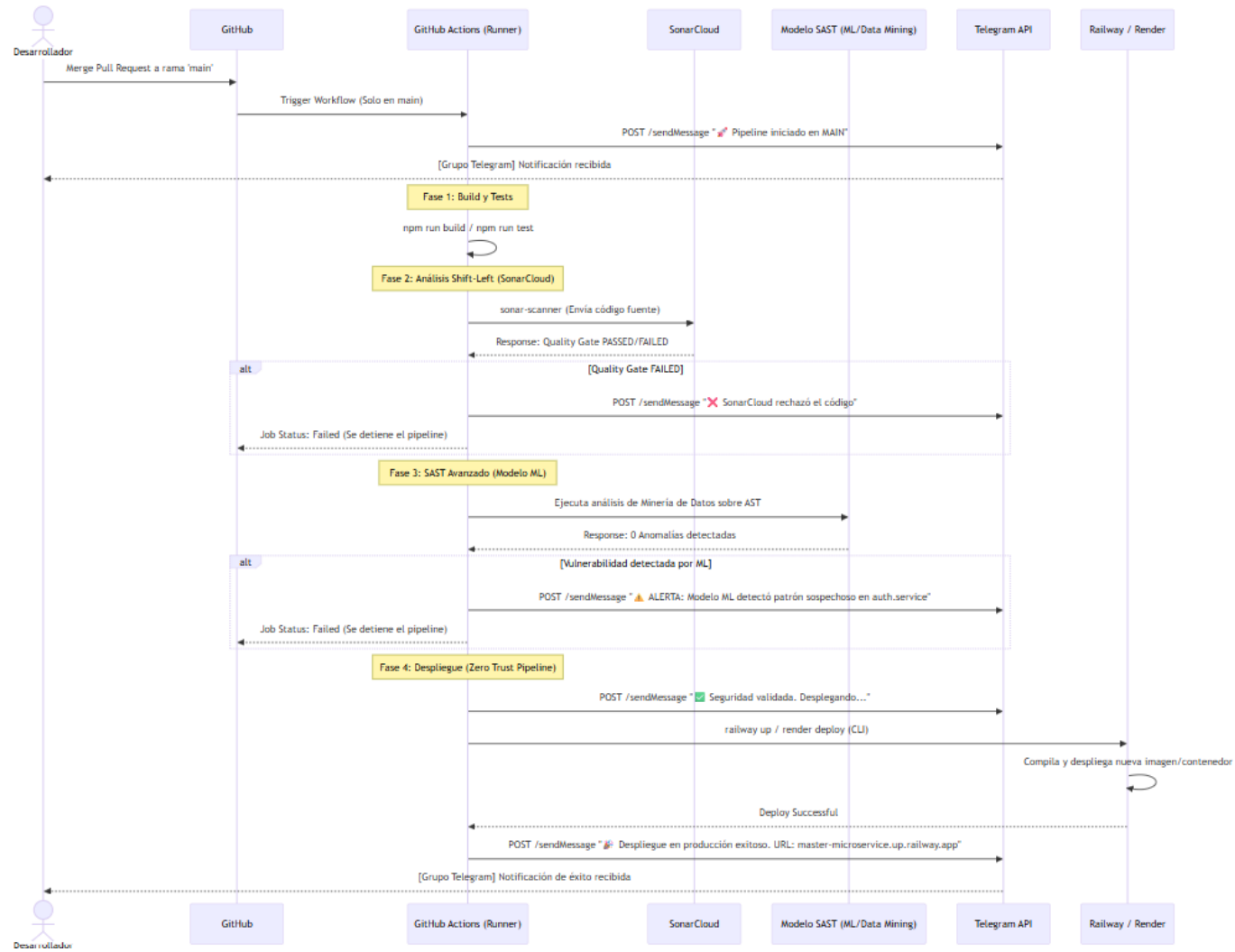
Eventos a notificar:

- Inicio del Pipeline en `main`.
- Éxito o fracaso del *Quality Gate* de SonarCloud.
- Alertas críticas si el Modelo de Minería de Datos detecta patrones sospechosos de vulnerabilidades.
- Estado del despliegue en Railway/Render (Éxito o Fallo).
- Merges exitosos hacia las ramas `dev` y `test`.

Diagrama de Secuencia: Pipeline CI/CD y Despliegue

El siguiente diagrama ilustra el comportamiento automatizado cuando un desarrollador mergea código a la rama `main`:

Figura 4: Diagrama de secuencia



Consideraciones técnicas para la implementación del Pipeline

- **Sobre el Modelo de Minería de Datos:** Dado que entrenar un modelo desde cero está fuera del alcance ágil del desarrollo de la app, se debe utilizar un enfoque pragmático: integrar una herramienta open-source basada en Machine Learning o un script custom que consuma una API de un modelo tipo *CodeBERT* fine-tuneado con datasets de CWEs (*Common Weakness Enumerations*). El script debe leer los archivos `.ts` o `.py` modificados en el commit y retornar un código de salida 0 (seguro) o 1 (vulnerable).
- **Limitaciones del PaaS Gratuito:** Es importante documentar en el repositorio que servicios gratuitos como Render "duermen" (*sleep*) tras 15 minutos de inactividad. La primera petición al microservicio maestro tras el reposo puede tardar ~ 30 segundos. Esto es aceptable para la fase de desarrollo del proyecto, pero el diseño de los microservicios hijos debe contemplar mecanismos de *Retry* (reintentos) en sus llamadas al Master para no fallar si el Master está despertando.
- **Seguridad de Secretos:** El token del Bot de Telegram, el token de SonarCloud y las credenciales de despliegue de Railway/Render deben estar estrictamente configurados en *Settings* ¿*Secrets and variables* ¿*Actions* en GitHub, aplicando el principio de mínimo privilegio.